

Online supplement:
A Reproducible Benchmark of Relational
Database Access-Layer
Performance in Express.js: A
Configuration-Specific Comparison
on PostgreSQL and MySQL

Mateusz Miotk

Faculty of Mathematics, Physics and Informatics, University of Gdańsk
`mateusz.miotk@ug.edu.pl`

This supplement carries measurement detail referenced from the main text. All tables are generated by the replication package (<https://github.com/mmiothk/express-db-access-performance>; archived at <https://doi.org/10.5281/zenodo.21359919>) from the same raw data as the main-text tables.

1 Dispersion on MySQL

The main text reports the coefficient-of-variation table for PostgreSQL and the per-engine maxima. Table S1 gives the full per-layer, per-pattern CV on MySQL across the 25 repeated runs (maximum 6.6%, `knex`/insert).

Table S1: Coefficient of variation (%) of throughput across the 25 repeated runs, per access layer and pattern on MySQL. Low values indicate stable measurements.

Layer	<code>point_read</code>	<code>range_scan</code>	<code>deep_fetch</code>	<code>aggregation</code>	<code>write</code>
<code>mysql2</code>	3.2	2.7	2.2	1.8	6.6
<code>knex</code>	2.4	3.0	2.2	2.6	6.6
<code>drizzle</code>	3.0	2.2	2.4	2.2	5.2
<code>prisma</code>	4.2	2.0	1.0	4.0	5.7
<code>sequelize</code>	2.2	2.0	1.9	2.1	5.9
<code>typeorm</code>	2.5	1.9	3.0	2.5	6.0
<code>objection</code>	3.3	3.4	2.5	2.5	5.9
<code>mikroorm</code>	3.5	2.9	3.0	1.9	4.9

2 Idiomatic round-trip counts

Table S2 reports, for each layer, the number of SQL statements the idiomatic deep fetch issues per request, captured from server-side statement logs. The counts motivate the main text’s observation that round-trip count alone does not determine throughput: the two layers issuing four statements (Prisma, Objection) sit at opposite ends of the throughput ladder.

Table S2: Number of SQL round trips each access layer issues per request on PostgreSQL, captured from server-side statement logging. The native drivers, Knex, and Drizzle issue a two-query deep fetch; the data-mapper ORMs’ eager-loading strategies choose their own counts. Full generated SQL and EXPLAIN plans are in the replication package.

Layer	Point read	Range scan	Deep fetch	Aggregation	Insert
pg	1	1	2	1	1
knex	1	1	2	1	1
drizzle	1	1	2	1	1
prisma	1	1	4	1	1
sequelize	1	1	1	1	1
typeorm	1	1	2	1	2
objection	1	1	4	1	1
mikroorm	1	1	1	1	1

3 Write throughput under default versus relaxed durability

Table S3 gives the per-layer insert throughput under each engine’s default durability configuration (the paper’s primary write regime) and under the symmetric relaxed regime (secondary, 10 replicates), with the relaxed÷default ratio quoted in the main text.

4 Equal-CPU-budget control

Table S4 gives the deep-fetch throughput with the server confined to 1, 2, or 4 cores (database and load generator pinned to disjoint cores); all nine values are quoted in the main text.

5 Application-tier versus end-to-end CPU efficiency

Table S5 reports, per layer on the deep/nested fetch, the median application-process-tree CPU and database-server CPU, the combined core-count, and

Table S3: Insert throughput (req/s) under the default durability configuration (PostgreSQL `fsync=on`, `synchronous_commit=on`; MySQL `innodb_flush_log_at_trx_commit=1`, `sync_binlog=1`; median of 25 replicates) and under the relaxed regime (all of the above off; median of 10), with the relaxed÷default ratio. At 50 concurrent connections, group commit amortizes the per-commit flushes: relaxing durability buys at most 14% on PostgreSQL (and $\leq 4\%$ on the layer-bound ORMs) and 19–24% on MySQL.

Layer	PostgreSQL			MySQL		
	default	relaxed	ratio	default	relaxed	ratio
pg	6064	6895	1.14	–	–	–
mysql2	–	–	–	809	966	1.19
pg-tuned	6280	7150	1.14	–	–	–
mysql2-tuned	–	–	–	808	972	1.20
knex	4693	5212	1.11	795	948	1.19
drizzle	4230	4675	1.11	796	950	1.19
prisma	5465	5675	1.04	714	879	1.23
sequelize	2668	2653	0.99	772	925	1.20
typeorm	2595	2675	1.03	732	886	1.21
objection	3926	4245	1.08	787	944	1.20
mikroorm	1571	1607	1.02	663	822	1.24

Table S4: Deep-fetch throughput (req/s, PostgreSQL, median of 3) with the server process confined by `taskset` to 1, 2, or 4 cores (database and load generator pinned to disjoint cores). `pg` reaches full throughput on a single core; Prisma requires roughly four to approach it, quantifying the CPU subsidy behind its near-native throughput.

Layer	1 core	2 cores	4 cores
pg	3663	3277	3700
prisma	896	1726	3025
mikroorm	463	577	542

throughput per application-CPU-second and per combined (application+database) CPU-second. The application-only figure understates the compute of layers that delegate more work to the database: `pg` leads application-tier efficiency by roughly $5\times$ over Prisma (3,741 vs. 740 req per app-CPU-s) but only $1.4\times$ end-to-end (813 vs. 596 req per combined-CPU-s), because it pushes about 3.6 database cores of work that the application-only metric ignores.

Table S5: Application-tier versus end-to-end CPU efficiency on the deep/nested fetch (PostgreSQL): median CPU (% of one logical core) of the Node.js process tree and of the database server, the combined core-count, and throughput per application-CPU-second and per combined (app+database) CPU-second. The application-only figure understates the compute of layers that delegate more work to the database; the combined figure is the end-to-end view. Load-generator CPU is excluded.

Layer	app CPU%	db CPU%	comb. cores	req/app-CPU-s	req/comb-CPU-s
pg	100	360	4.60	3741	813
knex	100	245	3.45	2491	722
drizzle	100	205	3.05	2080	682
prisma	498	120	6.18	740	596
sequelize	100	50	1.50	989	659
typeorm	105	120	2.25	1264	590
objection	100	50	1.50	1037	691
mikroorm	100	45	1.45	571	394

6 Coordinated-omission-corrected open-loop sweep

Table S6 gives the full constant-arrival sweep (PostgreSQL, five layers, offered rates 250–4,000 requests/s) behind the open-loop paragraph of the main text: achieved throughput, corrected latency percentiles measured from each request’s *intended* send time, and timeout counts (10 s cap, clipped into the distribution).

7 Pool-size sensitivity

The primary campaign fixes every layer’s connection pool at 10 to isolate the access layer from pool tuning. Table S7 varies the pool from 1 to 50 for a representative layer of each tier on the deep fetch (PostgreSQL, 50 connections). The three-band ordering (pg/Prisma, then Knex, then MikroORM) is preserved at every pool size, and each layer’s throughput saturates at a pool well below 50, so the fixed-pool choice does not drive the ranking; a very small pool (1–2) is the one regime that compresses the differences, because every layer then queues on connection acquisition.

Table S6: Open-loop tail latency, corrected for coordinated omission: p99 (ms) on the deep/nested fetch (PostgreSQL) under a constant offered request rate, with every latency measured from the request’s intended start time and timed-out requests included at their clipped value. [†]marks cells that did not sustain the offered rate (achieved<95% or any timeouts).

Layer	250/s	500/s	1000/s	2000/s	4000/s
pg	2	2	2	2	1201 [†]
prisma	3	2	2	4	2110 [†]
knex	2	2	2	33	9420 [†]
sequelize	2	3	41	10757 [†]	10944 [†]
mikroorm	3	15	11220 [†]	11399 [†]	11878 [†]

Table S7: Deep-fetch throughput (req/s, PostgreSQL, 50 connections, median of 3) as the connection pool varies from 1 to 50, per access layer. The primary campaign fixes the pool at 10; each layer saturates at a pool well below 50 and the ordering is preserved at every pool size, so the fixed-pool choice does not drive the ranking.

Layer	1	2	5	10	20	50
pg	923	1755	3571	3771	3800	3494
knex	764	1422	2419	2493	2533	2386
drizzle	895	1581	2023	2007	2085	1952
prisma	962	1717	3514	3679	3702	3491
typeorm	668	1068	1226	1288	1282	1196
mikroorm	543	550	553	542	539	514

8 Transactional multi-statement write

Beyond the single-row insert of the primary matrix, Table S8 reports a transactional write, `POST /threads`, which inserts a post and five comments in one transaction through each layer’s transaction facility, for a representative layer of each tier on PostgreSQL under default durability. The full layer $\times$ engine matrix for this pattern is left to future work. Because the transaction commits once, its per-commit fsync dominates and the layers cluster much more tightly than the between-layer spread on reads; the ordering tracks the single-row insert, so the write conclusions extend to a multi-statement transactional write.

Table S8: Transactional write throughput (req/s) and tail latency (p99, ms) for `POST /threads`, which inserts a post and five comments in a single transaction, on PostgreSQL under default durability (50 connections, median of 5). A representative layer of each tier is shown; the full matrix is left to future work. Between-layer spread is $3.3\times$ (Prisma 2412 down to MikroORM 740), well below the $6.55\times$ read spread, and the ordering tracks the single-row insert, so the write conclusions extend to a multi-statement transactional write.

Layer	req/s	p99 (ms)
<code>pg</code>	2347	44
<code>knex</code>	2029	49
<code>prisma</code>	2412	41
<code>typeorm</code>	1790	48
<code>mikroorm</code>	740	90

9 Dispersion on PostgreSQL

Table S9 gives the full per-layer, per-pattern coefficient of variation of throughput on PostgreSQL across the 25 repeated runs (maximum 7.6%, `pg`/insert), the companion to the MySQL table in Section S1.

10 Resource footprint

Table S10 gives the per-layer application-process CPU (median % of one logical core) and peak resident memory on the deep/nested fetch (PostgreSQL), the detail behind the CPU trade-off discussed in the main text (Prisma $\approx 498\%$, the other layers $\approx 100\%$; peak RSS 204–271 MB).

Table S9: Coefficient of variation (%) of throughput across the 25 repeated runs, per access layer and pattern on PostgreSQL. Low values indicate stable measurements.

Layer	point_read	range_scan	deep_fetch	aggregation	write
pg	4.3	2.9	2.6	1.9	7.6
knex	2.6	3.1	3.2	3.8	5.5
drizzle	2.5	2.6	2.9	2.8	3.6
prisma	3.6	3.4	3.6	4.3	4.3
sequelize	3.1	2.5	2.2	1.7	3.8
typeorm	2.8	2.6	2.0	2.6	3.8
objection	3.9	3.0	2.9	2.7	3.4
mikroorm	2.7	2.4	2.9	2.9	3.7

Table S10: Server-process resource footprint on the deep/nested fetch (PostgreSQL): median CPU utilization (% of one core) and peak resident memory (MB) of the Node.js process under load. The load generator and database run in separate processes.

Layer	Category	CPU (%)	RSS (MB)
pg	native-driver	100	205
knex	query-builder	100	204
drizzle	orm-lightweight	100	250
prisma	orm	498	266
sequelize	orm	100	248
typeorm	orm	105	271
objection	orm	100	211
mikroorm	orm	100	255

11 Pinned versions

Table S11 lists the pinned versions of every evaluated access layer and tool, recorded from the logfile on the measurement date.

Table S11: Pinned versions of the evaluated access layers and tooling.

Layer / tool	Version	Layer / tool	Version
pg	8.22.0	sequelize	6.37.8
mysql2	3.22.5	typeorm	0.3.30
knex	3.3.0	objection	3.1.5
drizzle-orm	0.36.4	@mikro-orm/core	6.6.15
@prisma/client	5.22.0	express	4.22.2
autocannon	7.15.0	PostgreSQL / MySQL	18.4 / 9.7.1

12 Per-pattern detail: the read patterns

The two cheapest read patterns show the smallest between-layer spread and are summarized in the main text; their full throughput and p99 tables are given here. Table S12 reports the point read (primary-key lookup) and Table S13 the keyset range scan, each by access layer and engine. The consequential patterns (deep/nested fetch, aggregation, insert) are tabulated in the main text.

Table S12: Point read: throughput (req/s, higher is better; median with 95% bootstrap confidence interval over the 25 repeated runs) and response-time p99 (ms, lower is better) by access layer and engine.

Layer	Category	PostgreSQL		MySQL	
		req/s [95% CI]	p99	req/s [95% CI]	p99
pg	native-driver	6695 [6527–6894]	9	–	–
mysql2	native-driver	–	–	4337 [4288–4431]	16
pg-tuned	native-tuned	6851 [6729–6941]	9	–	–
mysql2-tuned	native-tuned	–	–	4302 [4137–4337]	14
knex	query-builder	5096 [5013–5132]	12	3837 [3769–3862]	15
drizzle	orm-lightweight	4555 [4505–4610]	13	3168 [3131–3183]	20
prisma	orm	6051 [5976–6094]	12	5993 [5946–6091]	11
sequelize	orm	3532 [3425–3560]	16	2746 [2731–2774]	21
typeorm	orm	4331 [4262–4403]	14	3517 [3469–3525]	16
objection	orm	4070 [3981–4125]	15	3222 [3156–3272]	18
mikroorm	orm	2185 [2164–2212]	26	1839 [1808–1879]	31

Table S13: Keyset range scan: throughput (req/s, higher is better; median with 95% bootstrap confidence interval over the 25 repeated runs) and response-time p99 (ms, lower is better) by access layer and engine.

Layer	Category	PostgreSQL		MySQL	
		req/s [95% CI]	p99	req/s [95% CI]	p99
pg	native-driver	3782 [3685–3823]	18	–	–
mysql2	native-driver	–	–	3231 [3150–3279]	21
pg-tuned	native-tuned	3811 [3763–3836]	17	–	–
mysql2-tuned	native-tuned	–	–	3216 [3186–3234]	18
knex	query-builder	3163 [3127–3177]	19	2952 [2917–3002]	20
drizzle	orm-lightweight	2905 [2874–2952]	20	2265 [2243–2299]	28
prisma	orm	3963 [3941–3995]	17	3936 [3888–3956]	18
sequelize	orm	2426 [2403–2435]	24	2074 [2060–2094]	27
typeorm	orm	2546 [2494–2582]	25	2474 [2447–2489]	24
objection	orm	2669 [2606–2698]	22	2514 [2466–2533]	23
mikroorm	orm	773 [759–776]	74	729 [716–736]	79

13 Same-SQL control: full table

Table S14 gives the full same-SQL (raw-path) control on the deep fetch for both engines: throughput when every layer executes the identical control statement (verified byte-identical by server-side capture). The main text reports the resulting collapse toward parity and reads it as a bound on the residual same-SQL difference, a compound intervention that isolates no single factor.

14 Per-pattern detail: aggregation

Table S15 gives the full aggregation-pattern throughput and p99 by access layer and engine. RQ3 finds aggregation the least layer-sensitive pattern once every layer runs the same raw correlated-subquery plan; the main text reports the ranking and the leading figures.

15 Idiomatic deep-fetch choices per access layer

Table S16 documents, for each access layer, the exact eager-loading API used on the deep/nested fetch, the number of SQL round trips it issues, the documented alternative loading strategy we did *not* use, and the query-logging state. Every layer uses its documented default (idiomatic) path; query logging is disabled and verified on all eleven adapters, and no lifecycle hooks or validation run on the read path. The six data-mapper and ORM layers split both ways on the join-versus-select-in default: Sequelize, TypeORM, and

Table S14: Bounding the residual same-SQL difference on the deep-fetch spread: throughput (req/s) of the idiomatic deep fetch versus the *same-SQL* control (median of 10 runs), in which every layer executes the identical two-statement SQL with identical row mapping through its raw-SQL facility. The ratio (idiomatic $\div$ same-SQL) measures the gap between each layer’s idiomatic relational-fetch path and the common raw-SQL path, a composite of query strategy, query count, protocol, the raw versus ORM API, and result marshalling and hydration changed together; the residual native-relative same-SQL spread ($1.47\times$ on PostgreSQL, $1.27\times$ on MySQL) bounds that compound difference without isolating any single factor.

Layer	PostgreSQL			MySQL		
	idiom.	same-SQL	ratio	idiom.	same-SQL	ratio
pg	3741	3643	1.03	–	–	–
mysql2	–	–	–	2405	2419	0.99
knex	2491	2906	0.86	2050	2305	0.89
drizzle	2080	3446	0.60	1446	2105	0.69
prisma	3685	4342	0.85	3701	4220	0.88
sequelize	989	2472	0.40	893	1912	0.47
typeorm	1327	2974	0.45	1170	2517	0.46
objection	1037	2845	0.36	860	2243	0.38
mikroorm	571	2869	0.20	535	2172	0.25

Table S15: Aggregation: throughput (req/s, higher is better; median with 95% bootstrap confidence interval over the 25 repeated runs) and response-time p99 (ms, lower is better) by access layer and engine.

Layer	Category	PostgreSQL		MySQL	
		req/s [95% CI]	p99	req/s [95% CI]	p99
pg	native-driver	7035 [6961–7116]	10	–	–
mysql2	native-driver	–	–	4195 [4168–4241]	17
pg-tuned	native-tuned	7731 [7538–7838]	9	–	–
mysql2-tuned	native-tuned	–	–	4194 [4132–4233]	17
knex	query-builder	5754 [5606–5837]	12	3945 [3908–3986]	17
drizzle	orm-lightweight	6427 [6347–6484]	11	3944 [3864–3974]	18
prisma	orm	7893 [7795–7994]	9	7383 [7348–7404]	9
sequelize	orm	4807 [4744–4831]	15	3390 [3352–3461]	19
typeorm	orm	6127 [6052–6157]	12	4371 [4289–4430]	17
objection	orm	3775 [3718–3820]	18	2963 [2928–2992]	22
mikroorm	orm	5279 [5225–5334]	14	3590 [3537–3627]	19

MikroORM default to a single join, while Prisma and Objection default to separate batched queries. The loading-strategy sensitivity check reported in the main text (Table S18) switches the join-default layers to select-in where that alternative is a byte-identical drop-in. Full per-adapter detail is in the replication package’s `METHODOLOGY.md`.

Table S16: Idiomatic deep-fetch implementation per access layer: the eager-loading API actually used, the SQL round trips it issues (measured from server-side statement logging for the portable layers; by construction for the native variants), the documented alternative loading strategy we did *not* use, and the query-logging state. Query logging is disabled everywhere; no lifecycle hooks or validation run on the read path (details in the replication package’s `METHODOLOGY.md`).

Layer	Deep-fetch API	Round-trips	Alternative not used	Logging
pg	hand-written JOIN $\times 2$	2	n/a (hand-written SQL)	off
mysql2	hand-written JOIN $\times 2$	2	n/a (hand-written SQL)	off
pg-tuned	named-prepared JOIN $\times 2$	2	n/a (hand-written SQL)	off
mysql2-tuned	execute() JOIN $\times 2$	2	n/a (hand-written SQL)	off
knex	builder .join() $\times 2$	2	n/a (hand-written SQL)	off
drizzle	builder .innerJoin() $\times 2$	2	relational query API (db.query)	off
prisma	include:	4	relationLoadStrategy: 'join'	off
sequelize	include: (separate:false)	1	separate:true (select-in)	off
typeorm	relations:	2	relationLoadStrategy: 'query'	off
objection	.withGraphFetched()	4	.withGraphJoined()	off
mikroorm	populate:	1	strategy: 'select-in'	off

16 Open-loop tail latency on MySQL

Table S17 is the MySQL companion to the PostgreSQL open-loop sweep (Supplement Table S6): coordinated-omission-corrected p99 on the deep/nested fetch at a constant offered rate. The sub-saturation tail is flat and small on both engines, and each layer collapses once the offered rate crosses its capacity, so the saturated-tail ordering holds on MySQL as on PostgreSQL.

17 Alternative eager-loading sensitivity

Table S18 reports the loading-strategy sensitivity check: for the data-mapper ORMs whose alternative strategy is a byte-identical drop-in, idiomatic deep-fetch throughput versus the alternative, on both engines. Both endpoints were verified to return byte-identical responses before timing.

18 MySQL insert commit-flush mechanism

Table S19 gives the direct `performance_schema` evidence behind the MySQL insert bottleneck: sampling the redo-log and binary-log flush wait instru-

Table S17: Open-loop tail latency on **MySQL**, corrected for coordinated omission: p99 (ms) on the deep/nested fetch under a constant offered request rate, every latency measured from the intended start time and timed-out requests clipped into the distribution. [†]marks cells that did not sustain the offered rate. As on PostgreSQL (Supplement Table S6), the sub-saturation tail is flat and small and each layer collapses once the offered rate crosses its capacity, so the saturated-tail ordering is engine-robust.

Layer	250/s	500/s	1000/s	2000/s	4000/s
mysql2	2	2	4	14	9658 [†]
prisma	3	3	3	4	2006 [†]
knex	2	2	2	297	10900 [†]
sequelize	3	3	2334 [†]	10814 [†]	10945 [†]
mikroorm	4	34	11269 [†]	11673 [†]	11800 [†]

Table S18: Alternative eager-loading sensitivity on the deep/nested fetch: idiomatic throughput (req/s) versus the alternative loading strategy, both engines, for the layers whose alternative is a byte-identical drop-in. Switching Sequelize and MikroORM from their idiomatic single join to select-in makes them slower on both engines (0.68–0.75 $\times$), so their deep-fetch deficit is not an artifact of a poor default. Objection is the converse: switching its idiomatic batched select-in to a single join (**withGraphJoined**) is byte-identical only on MySQL, where it is 1.6 $\times$ faster, so part of Objection’s MySQL deficit *is* a default-strategy choice. **typeorm** is omitted (its query strategy errors on the ordered nested **findOne** in the pinned version, itself a portability caveat), as is Objection on PostgreSQL, where **withGraphJoined** did not return a byte-identical result. The idiomatic column is re-measured in the same paired run (five replicates) as its alternative, so its absolute values differ from Table ?? by day-to-day drift; the idiomatic-versus-alternative ratio is the comparison of interest.

Layer	Switch	PostgreSQL		MySQL	
		idiom.	alt.	idiom.	alt.
sequelize	join→select-in	897	668	788	590
mikroorm	join→select-in	421	296	423	289
objection	select-in→join	–	–	802	1277

ments around a fixed insert workload at default durability, the flush is a shared engine floor—similar per-insert across the native driver, query builder, and ORM despite their very different read performance—and relaxing durability removes it.

Table S19: MySQL insert commit-flush mechanism (`performance_schema` wait instruments, default durability `innodb_flush_log_at_trx_commit=1`, `sync_binlog=1`): per-insert wall time, the redo-log+binlog flush wait per insert, and the flush share of wall time, over 4000 inserts per layer. The flush wait is a shared engine floor – similar across layers despite their very different read performance – so it caps insert throughput regardless of the access layer. Relaxing durability (`mysql2`, `trx_commit=0`, `sync_binlog=0`) removes the flush and lifts throughput to 787 req/s, confirming the floor.

Layer (MySQL)	req/s	per-insert (ms)	flush/insert (ms)	flush share
<code>mysql2</code>	670	1.493	0.588	39%
<code>knex</code>	671	1.489	0.571	38%
<code>mikroorm</code>	588	1.699	0.686	40%
<i>Relaxed durability (control):</i>				
<code>mysql2*</code>	787	1.27	0.011	1%

19 Post-restart environmental-robustness check

Table S20 reports the environmental-robustness check: after a full restart of both database engines, a representative deep-fetch subset re-measured eight days after the primary campaign. The layer ranking reproduces on both engines (`pg` > Prisma >> MikroORM on PostgreSQL; Prisma > `mysql2` >> MikroORM on MySQL); absolute throughput is 3–23% lower, the day-to-day host drift the paper’s relative analysis is designed to absorb. Because PostgreSQL and MySQL cells are not measured simultaneously, this drift is non-uniform across engines (3% on `mysql2`, 13% on `pg`, 23% on MikroORM/PostgreSQL), so the cross-engine interaction ratios could in principle carry some residual drift; they are far larger than 23% (reads 1.0–1.6, writes 2.4–7.7), so the engine-dependence conclusion holds.

20 Utilization-controlled tail latency

Tables S21 and S22 give the coordinated-omission-corrected p99 on the deep fetch when each layer is offered 50/70/85/95% of *its own* saturating throughput (its primary closed-loop deep-fetch rate), replicated five times, on both engines. At matched utilization the tails are small and similar across layers—for example on PostgreSQL every layer holds p99 near 2–4 ms at 50%—so

Table S20: Environmental-robustness check (review 6.7): after a full restart of both database engines (cold buffer pools, fresh connections), deep-fetch throughput of a representative subset re-measured eight days after the primary campaign, against the primary median. The layer ranking reproduces on both engines; absolute throughput drifts with host conditions (larger on PostgreSQL), which is exactly the day-to-day variation the paper’s *relative* analysis is designed to absorb. Single host, so this checks temporal robustness, not cross-machine generalization.

Engine	Layer	primary req/s	post-restart req/s	ratio
PostgreSQL	pg	3741	3270.5	0.87
PostgreSQL	prisma	3685	3211	0.87
PostgreSQL	mikroorm	571	437.5	0.77
MySQL	mysql2	2405	2331	0.97
MySQL	prisma	3701	3161	0.85
MySQL	mikroorm	535	430.5	0.81

the wide saturated-p99 ladder in the main text is a capacity-and-queueing effect, not a difference in intrinsic tail latency; the tail rises only as a layer nears its own capacity. This is the near-intrinsic latency companion to the saturated primary p99. The 50/70% cells, which carry that comparison, are stable across runs; the 85/95% cells are increasingly noisy (p99 varies several-fold between runs as a layer approaches saturation), so those columns are read as trend, not precise values.

21 Multi-worker scaling

Table S23 scales each of a representative subset to 1, 2, and 4 Express workers (a shared-port `node` cluster) so every layer can use the cores that Prisma’s in-process engine already consumes at one worker. At one worker the native driver and Prisma are near parity; as workers consume the idle cores the single-pool layers scale near-linearly and the native driver overtakes Prisma (1.6× on PostgreSQL, 1.4× on MySQL at four workers). Prisma’s parity is therefore specific to a single-worker deployment—the direct test of the single-host resource-competition concern.

22 Pool-size frontier on MySQL

Table S24 is the MySQL companion to the PostgreSQL pool sweep (Supplement Table S7): deep-fetch throughput as the connection pool varies from 1 to 50, per layer. Each layer saturates at a pool well below 50 and the ordering is preserved at every pool size, so the fixed pool of ten does not

Table S21: Utilization-controlled open-loop tail on the deep fetch (PostgreSQL): coordinated-omission-corrected p99 (ms, median of five runs) when each layer is offered a fixed fraction of *its own* saturating throughput. At matched utilization the tails are small and similar across layers—the large saturated p99 gap is a capacity/queueing effect, not intrinsic tail latency; the tail rises only as each layer approaches its own capacity.

Layer	50%	70%	85%	95%
pg	2.1	4.6	18	83
knex	2.3	4.9	25.3	26.2
drizzle	2.5	2.9	18.4	15.9
prisma	3.4	6.9	11.7	16.3
sequelize	2.9	3	8	12.7
typeorm	3.2	4.2	17.3	27
objection	3	3.3	7.2	17.2
mikroorm	3.4	3.7	4.3	7.7

Table S22: Utilization-controlled open-loop tail on the deep fetch (MySQL): coordinated-omission-corrected p99 (ms, median of five runs) when each layer is offered a fixed fraction of *its own* saturating throughput. At matched utilization the tails are small and similar across layers—the large saturated p99 gap is a capacity/queueing effect, not intrinsic tail latency; the tail rises only as each layer approaches its own capacity.

Layer	50%	70%	85%	95%
mysql2	4.6	6.8	10	33.2
knex	2	2.1	36.2	97.4
drizzle	4.2	5.2	13.1	25.7
prisma	3.5	6.6	15.6	19.4
sequelize	3.1	3	7.1	19
typeorm	2.5	3.6	6.9	17.3
objection	3.1	3.1	6.5	26.7
mikroorm	3.7	3.5	7.4	10.6

Table S23: Multi-worker deep-fetch throughput (req/s, median of three runs) as each layer is scaled to 1, 2, and 4 Express workers (node cluster, shared port). At one worker the native driver and Prisma are near parity; as workers consume the otherwise-idle cores the single-connection-pool layers scale near-linearly and the native driver overtakes Prisma (whose in-process engine already used several cores at one worker), so Prisma’s parity is specific to a single-worker deployment.

Layer	1 worker	2 workers	4 workers
<i>PostgreSQL</i>			
pg	3308	6554	10178
prisma	3141	5861	6439
mikroorm	431	869	1748
<i>MySQL</i>			
mysql2	2126	4559	8890
prisma	3184	5742	6152
mikroorm	436	851	1706

drive the ranking on either engine.

Table S24: Deep-fetch throughput (req/s, MySQL, 50 connections, median of 3) as the connection pool varies from 1 to 50, per access layer. The primary campaign fixes the pool at 10; each layer saturates at a pool well below 50 and the ordering is preserved at every pool size, so the fixed-pool choice does not drive the ranking.

Layer	1	2	5	10	20	50
mysql2	1590	2192	2117	2451	2343	2412
knex	1268	1940	1984	2085	2120	2030
drizzle	1287	1365	1408	1442	1443	1316
prisma	901	1561	3299	3732	3698	3553
typeorm	736	1123	1126	1180	1147	1102
mikroorm	518	514	519	533	526	509

23 Mixed read/write workload

Table S25 interleaves the deep-fetch read with single-row inserts at 90:10 and 70:30 read:write, both engines, with a physical write reset per run. The layer ordering matches the read ordering and is insensitive to the mix, because the deep-fetch read dominates the cost; exploratory.

Table S25: Mixed read/write workload: throughput (req/s) and p99 (ms) under an autocannon request mix interleaving the deep-fetch read with single-row inserts, at 90:10 and 70:30 read:write, both engines, median of five runs with a physical write reset per run. The layer ordering matches the read ordering and is insensitive to the mix (the deep-fetch read dominates); exploratory.

Engine	Layer	read:write	req/s	p99
PG	pg	90:10	4203	17
PG	pg	70:30	4223	17
PG	knex	90:10	2920	24
PG	knex	70:30	2889	24
PG	prisma	90:10	3954	17
PG	prisma	70:30	3903	18
PG	mikroorm	90:10	668	96
PG	mikroorm	70:30	681	96
MySQL	mysql2	90:10	2481	38
MySQL	mysql2	70:30	2568	34
MySQL	knex	90:10	2259	35
MySQL	knex	70:30	2249	35
MySQL	prisma	90:10	2625	33
MySQL	prisma	70:30	2633	40
MySQL	mikroorm	90:10	603	115
MySQL	mikroorm	70:30	635	102

24 Concurrency sweep

Figure S1 sweeps the deep/nested fetch from 1 to 256 client connections for a representative layer of each tier, both engines. The 50-connection operating point used throughout sits above the knee for every layer; the ordering is load-robust above about 32 connections. This is the evidence behind the choice of operating point.

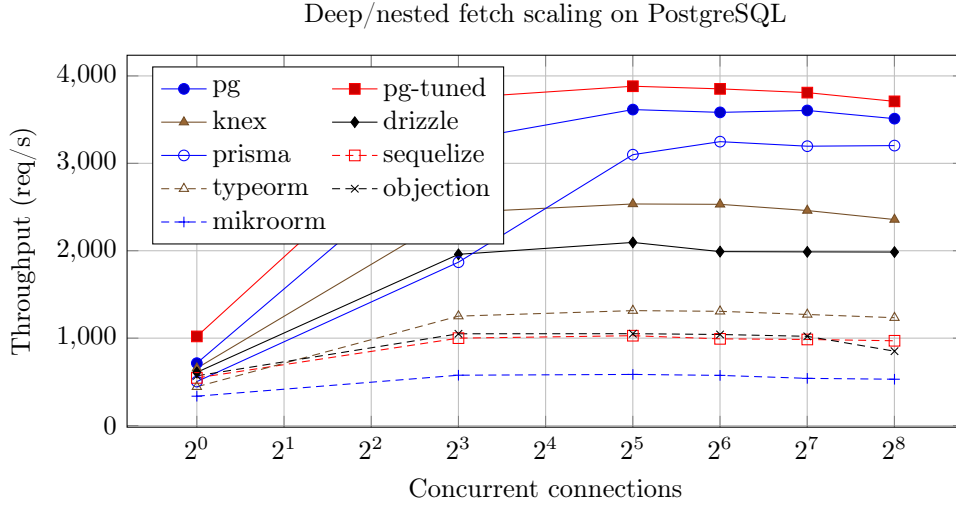


Figure S1: Throughput of each access layer on the deep/nested fetch as concurrency increases (postgres). The three-band ordering (native and tuned-native with Prisma on top, query builder and lightweight ORM in the middle, data-mapper ORMs below) holds at 32 connections and above; at 1 and 8 connections Prisma is mid-pack (Section ??).

25 Per-layer cross-engine ranks

Table S26 gives each portable layer’s throughput rank on PostgreSQL versus MySQL for the deep fetch and the insert, backing the rank correlations in the main text: the deep-fetch orderings are nearly identical ($\rho = 0.96$) while the insert orderings scramble ($\rho = 0.43$), most visibly Prisma (rank 1 on PostgreSQL, 6 on MySQL).

26 A reporting checklist for access-layer benchmarks

Distilled from the design decisions and the pitfalls this study confronted, the following items are the ones an access-layer benchmark should report so that its claims can be scoped and reproduced. It is a reporting aid, not a standard.

Table S26: Per-layer throughput rank (1 = fastest) on PostgreSQL versus MySQL for the seven portable layers, backing the rank correlations. On the deep/nested fetch the two engines’ orderings are nearly identical (Spearman $\rho = 0.96$); on the insert they scramble ($\rho = 0.43$), most visibly Prisma, which is fastest on PostgreSQL (rank 1) but sixth on MySQL. This is why the write ordering is reported as engine-dependent.

Layer	Deep fetch		Insert	
	PG	MySQL	PG	MySQL
knex	2	2	2	2
drizzle	3	3	3	1
prisma	1	1	1	6
sequelize	6	5	5	4
typeorm	4	4	6	5
objection	5	6	4	3
mikroorm	7	7	7	7

1. **Correctness before timing.** Assert byte-identical responses across layers (final serialized bodies), or state what differs; several defects in this study were caught only this way.
2. **Experimental unit.** Treat the freshly booted run, not the individual request, as the unit; requests within a run are not independent.
3. **Same-host repetition is not environmental replication.** Report how many runs, on how many hosts, over what period; do not call same-host repeats “independent.”
4. **Idiomatic treatment.** State the exact API, its documentation page and version, the declined alternative, and whether logging, hooks, validation, and identity maps are on.
5. **Latency construct.** Distinguish saturated closed-loop response time (which tracks throughput and queueing) from utilization-controlled or open-loop latency; report the operating point.
6. **Same-SQL / shared-plan control.** Report it as a compound intervention (it changes query formulation, protocol, preparation, round-trips, and mapping together), not a mechanism decomposition.
7. **Resource accounting.** Report application-tier and database CPU separately; throughput per CPU-second, not throughput alone.
8. **Write-state control.** Reset the mutable state between runs and state the durability regime (default versus relaxed).

9. **Provenance.** Pin engine images by digest and dependencies by lock-file; archive raw per-cell records, seeds, and a table-to-record manifest.
10. **Scope.** State the host topology, workload, versions, and dates, and confine claims to them.